(i) Algorithm 2 (Dynamic Programming)

We index the $\boldsymbol{n}$ distinct food portions as $\mathbf{1}, \ldots, \boldsymbol{n}$.

Subproblem (Recurrence Relation): Let $\boldsymbol{X}[\boldsymbol{k}, \boldsymbol{p}]$ represent the minimum caloric capacity required to construct a meal plan with a total protein content of at least $\boldsymbol{p}$. For this reason, a loop for **p = 1** to **P_total** is utilized, where $\boldsymbol{P}_{\textbf{total}}$ is the cumulative protein sum of all available portions. The input is restricted exclusively to the first $\boldsymbol{k}$ portions.

Boundary Conditions:

- If we target a protein requirement of 0 using 0 portions, we do not need to select any item, meaning the minimum calories required is 0 ($\boldsymbol{X}[\mathbf{0}, \mathbf{0}] = \mathbf{0}$).
- When evaluating 0 items for a target protein quantity $\boldsymbol{p} > \mathbf{0}$, we set $\boldsymbol{X}[\mathbf{0}, \boldsymbol{p}] = \infty$ to facilitate the **< min** comparison, as it is impossible to satisfy a positive protein requirement with zero items.
- Given the first $\boldsymbol{k}$ available portions and a target of $\boldsymbol{p} = \mathbf{0}$, the minimum calories required is trivially 0 ($\boldsymbol{X}[\boldsymbol{k}, \mathbf{0}] = \mathbf{0}$).

<u>When examining the $\boldsymbol{k}$-th portion, we consider two main cases depending on its protein content $\boldsymbol{P}_{\boldsymbol{k}}$:</u>

Case 1: $\boldsymbol{P}_{\boldsymbol{k}} < \boldsymbol{p}$ (where $\boldsymbol{p} \in [\mathbf{1}, \ldots, \boldsymbol{P}_{\textbf{total}}]$). We can either:

- ***Include it:*** Collect the remaining $\boldsymbol{p} - \boldsymbol{P}_{\boldsymbol{k}}$ protein from the first $\boldsymbol{k} - \mathbf{1}$ portions and add to **min**$\boldsymbol{X}$ the caloric cost $\boldsymbol{C}_{\boldsymbol{k}}$ of the $\boldsymbol{k}$-th portion.

- ***Exclude it:*** Collect the full protein quantity $\boldsymbol{p}$ (where $\boldsymbol{p} \in [\mathbf{1}, \ldots, \boldsymbol{P}_{\textbf{total}}]$) solely from the first $\boldsymbol{k} - \mathbf{1}$ portions.

Case 2: $\boldsymbol{P}_{\boldsymbol{k}} \geq \boldsymbol{p}$. We can either:

- ***Include it:*** Including the $\boldsymbol{k}$-th portion immediately satisfies the target protein requirement of at least $\boldsymbol{p}$. Thus, the remaining protein to be gathered from the first $\boldsymbol{k} - \mathbf{1}$ portions drops to zero, and we add $\boldsymbol{C}_{\boldsymbol{k}}$ to the cost (**minX**).

- ***Exclude it:*** We must collect the full protein quantity $\boldsymbol{p}$ from the first $\boldsymbol{k} - \mathbf{1}$ portions.

Recurrence Equation:

$$\mathbf{X}[\mathbf{k}, \mathbf{p}] = \begin{cases} \mathbf{0}, & \textbf{if } \mathbf{k} = \mathbf{0} \textbf{ and } \mathbf{p} = \mathbf{0} \\ \infty, & \textbf{if } \boldsymbol{P} = \mathbf{0} \\ \mathbf{min}\{\mathbf{X}[\mathbf{k}-\mathbf{1}, \mathbf{p}], \mathbf{X}[\mathbf{k}-\mathbf{1}, \mathbf{P_k}] + \mathbf{C_k}\} & \textbf{if } \mathbf{P_k} < \mathbf{p} \\ \mathbf{min}\{\mathbf{X}[\mathbf{k}-\mathbf{1}, \mathbf{p}], \mathbf{X}[\mathbf{k}-\mathbf{1}, \mathbf{0}] + \mathbf{C_k}\} & \textbf{if } \mathbf{P_k} \geq \mathbf{p} \end{cases}$$

Pseudocode:

```
P_total = 0
for i in P:
        P_total += i
X[0,0]=0
for p=1 to P_total:
        X[0,p]=∞
for k=1 to n:
        X[k,0]=0
        for p=1 to P_total:
                if P_k < p:
                        X[k,p] = min{X[k-1,p], X[k-1,p-P_k] + C_k}
                else :
                        X[k,p] = min{X[k-1,p], X[k-1,0] + C_k}
while(p>=0 && calories > W):
        protein = p;
        p--;
        calories = X[n][p];
return protein, calories
```

Complexity Analysis:

- 1 loop for i in P = **O(n)**
- 1 loop for p=1 to P_total = **O(n*P_{max})** (since the maximum possible value for $\boldsymbol{P_{total}}$ occurs when all portions contain the maximum individual protein value $\boldsymbol{P_{max}}$)
- 1 loop for k = 1 to n = O(n)
  1 n*ested* loop for p = 1 to P_{total} = O(P_total)
  } $O(n * P\_total)$ = O(n^2*P_{max})

  Overall time complexity: O(nP_total) = O(n^2P_{max})

(iii) EXPERIMENTAL CONCLUSIONS AND SUMMARY FROM ALGORITHMS EXECUTION

Time Complexity:

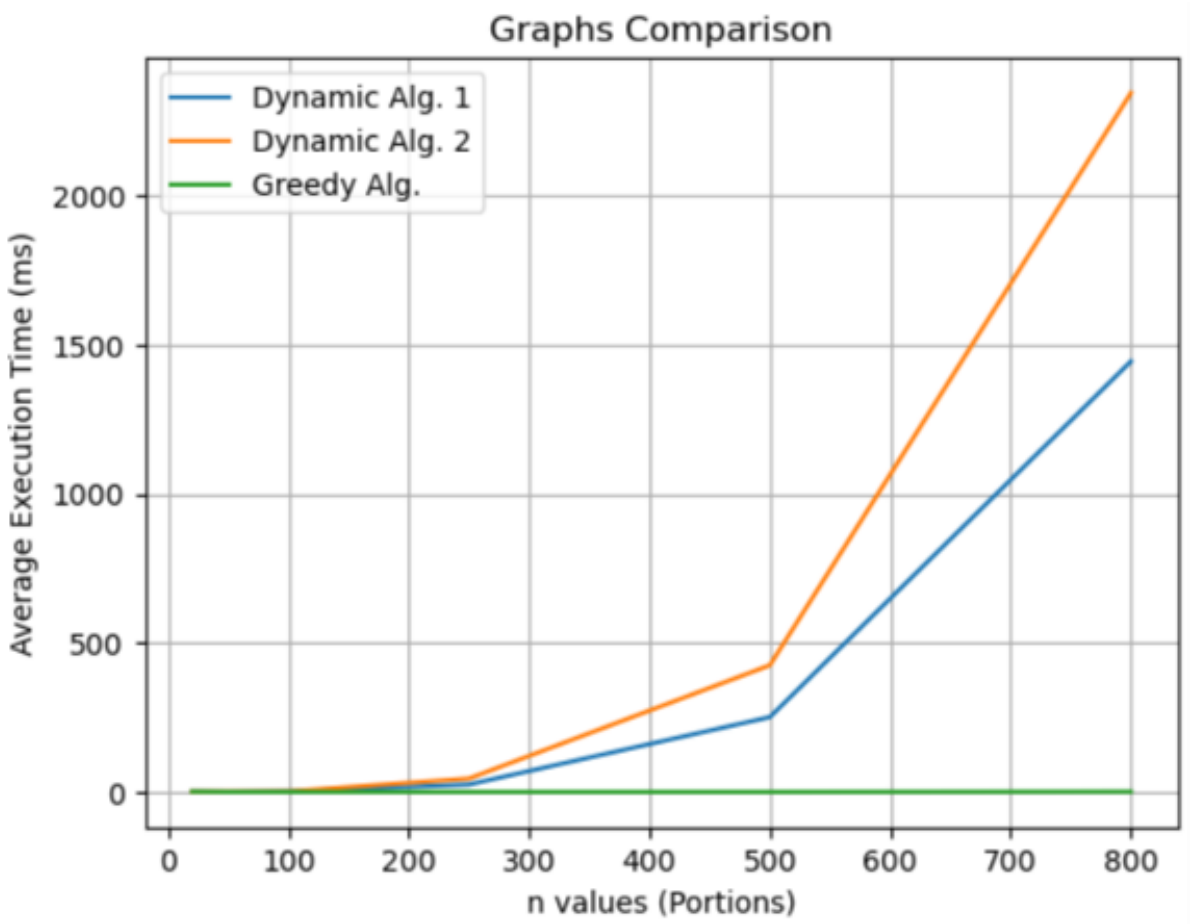
- **Algorithm 1 (Dynamic Programming):** This approach has a time complexity of $\boldsymbol{O(nW)}$, which is pseudo-polynomial (exponential relative to the input bit-length of $\boldsymbol{W}$, since if $\boldsymbol{b}$ bits store $\boldsymbol{W}$, then $\boldsymbol{W = 2^b}$). Experimentally, as $\boldsymbol{n}$ **(# of food portions)** increases, the target capacity $\boldsymbol{W}$ scales alongside it, causing the execution time to grow rapidly. For small values of $\boldsymbol{n}$, execution takes only 1 to 2 milliseconds, but once $\boldsymbol{n}$ approaches $\boldsymbol{500}$, the runtime jumps sharply to around $\boldsymbol{\sim 250}$ **ms**, as shown by the blue line in the chart.

- **Algorithm 2 (Dynamic Programming):** With a time complexity of $\boldsymbol{O(n^2 P_{max})}$, this algorithm behaves similarly to Algorithm 1 because $\boldsymbol{P_{max}}$ depends on the numeric scale of the input bits values. As shown by the orange line, the runtime for small inputs is near zero, but it shoots up even faster than Algorithm 1, reaching approximately $\boldsymbol{\sim 450}$ **ms** when $\boldsymbol{n}$ nears $\boldsymbol{500}$.

- **Algorithm 3 (Greedy):** With a time complexity of $\boldsymbol{O(n \log n)}$, the primary cost here is sorting the $\boldsymbol{n}$ portions. This is a true polynomial-time algorithm, resulting in a highly stable performance. On the graph, its runtime appears as a flat line hovering near 0 ms (the green line of the graph staying between $\boldsymbol{0}$ **and** $\boldsymbol{1}$ **ms**), showing only a negligible 1 ms increase even for massive values of $\boldsymbol{n}$.

Time complexity summary

As the number of portions $\boldsymbol{(n)}$ scales up, the dynamic programming approaches face an aggressive increase in execution time, with the second algorithm being the slowest. On the other hand, the greedy heuristic maintains near-instantaneous execution times close to 0 ms even under high workloads



Space Complexity:

Algorithms 1 & 2 (Dynamic Programming): With space complexities of $\boldsymbol{O(nW)}$ and $\boldsymbol{O(nP_{total})}$ respectively, their memory usage grows directly alongside their average execution time scaling.

For a small input size $\boldsymbol{(n = 20)}$

Algorithm 1 (with $\boldsymbol{\max(W) = 420}$)

- Matrix size: (n + 1) x (W + 1) = 21 x 421 = 8.841 cells.
- Max Memory: 8.841 x 4 bytes = 35.364 bytes

Algorithm 2 (with $\boldsymbol{\max(P_{total}) = 600}$)

- Matrix size: (n + 1) x (Ptotal + 1) = 21 x 601 = 12.621 cells.
- Max Memory: 12.621 x 4 bytes = 50.484 bytes

For a large input size $\boldsymbol{(n = 1000)}$**:**

Algorithm 1 (with $\boldsymbol{\max(W) = 1.050.000}$)

- Matrix size: (n + 1) x (W + 1) = 1001 x 1.050.001 = 1.051.051.001 cells
- Max Memory: 1.051.051.001 x 4 bytes = 4.204.204.004 bytes = 4,20 GB

Algorithm 2: (with $\boldsymbol{\max(P_{total}) = 1.500.000}$)

- Matrix size: (n + 1) x (Ptotal + 1) = 1001 x 1.500.001 = 1.501.501.001 cells
- Max Memory: 1.501.501.001 x 4 bytes = 6.006.004.004 bytes = 6,01 GB

- **Algorithm 3 (Greedy):** Possesses a space complexity of $\boldsymbol{O(n)}$**.** It only uses a linear array structure **(Java ArrayList)** to sort the entries, keeping memory allocation minimal (a few kilobytes) even for huge $\boldsymbol{n}$ values.

  Space complexity summary
  Memory allocation proved to be the primary bottleneck during our tests. For very large scales (e.g., $\boldsymbol{n > 950}$), the massive multi-gigabyte table allocations of the exact dynamic programming algorithms 1 & 2 triggered **out-of-memory heap space exceptions**, crashing the program. The greedy algorithm completely bypassed this limitation due to its minimal footprint.

Approximation Ratio ($\boldsymbol{SOL/OPT}$) and Trade-offs:

In theory, greedy heuristics do not guarantee an optimal solution compared to exact dynamic programming models. However, our benchmarks show that under realistic performance limits, exact models can easily overload standard system memory. Looking closely at the greedy algorithm's approximation ratio **($\boldsymbol{SOL/OPT}$)**, the solution quality steadily converges to 1 as the problem scale grows. At $\boldsymbol{n = 800,}$ the ratio reaches $\boldsymbol{0.9988}$, meaning the greedy solution is practically indistinguishable from the absolute mathematical optimum.

Final Conclusion:

For real-world software where an absolute, mathematically perfect optimum is not strictly mandatory -such as this meal planner- the greedy algorithm is a significantly better architectural choice. While dynamic programming algorithms consume immense system memory for negligible gains in solution quality, the greedy approach provides an ideal balance by offering blazing-fast performance and minimal resource costs with a nearly perfect approximation of the optimal result.